Accelerating Open-Source InSAR Time-Series Analysis on Consumer GPUs:

A PyTorch-Based Network Inversion Solver for MintPy

Syota Sasaki*
Earthsea Wizard Tech Lab, Japan · syota.sasaki@earthsea-wizard.com · *presenter

TASTI 2026 · TAINAN
Paper No.
T0043
Poster · 0201 Earth Observation and Remote Sensing

1 The bottleneck

InSAR time-series analysis tracks slow ground deformation over months to years: volcanic inflation, motion on active faults before and after earthquakes, land subsidence. Sentinel-1 and NISAR [1] archives outgrow the software that consumes them.

43.5 %

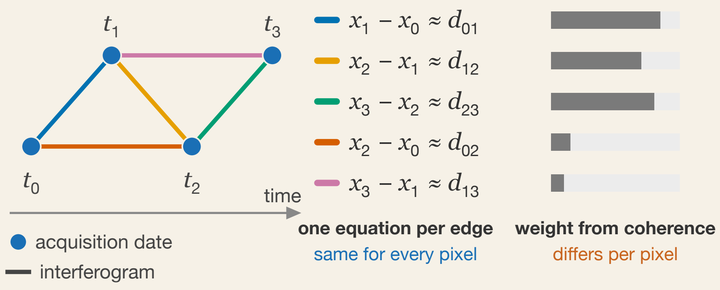
One step: the network inversion — the other 17 steps

Step times of a complete MintPy [2] run (all 18 steps of `smallbaselineApp`, Fernandina scene, on the CPU), as reported by MintPy with interpreter start-up excluded. The network inversion is a least-squares problem, solved pixel by pixel.

This work: that step now runs 44× faster on one consumer GeForce GPU (block 5), switched on by a single template option.

2 Why a least-squares inversion?

- An *interferogram*, the phase difference of two radar images of the same ground, measures how far it moved toward the satellite between the two dates, to millimetres.
- Distant pairs lose coherence, so many short pairs form a *network*: dates as vertices, interferograms as edges.

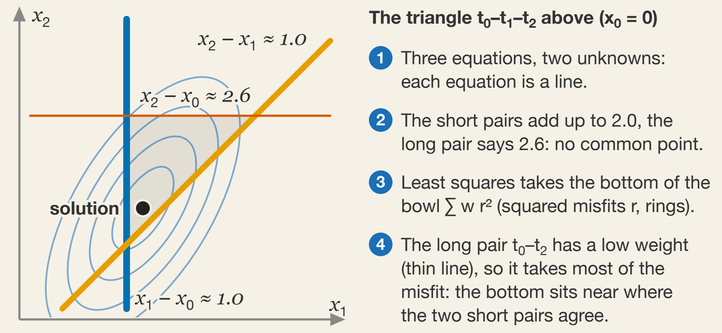


x_i: displacement at date t_i, with $x_0 = 0$ as the reference.

Why not just solve them? Moving from t_0 to t_1 and then on to t_2 is the same as moving from t_0 to t_2, so the three interferograms should satisfy

$$d_{01} + d_{12} = d_{02}.$$

Real interferograms each carry their own noise and never add up exactly. No displacement history fits them all, so the best compromise has to be found by optimization.



One pixel, illustrative numbers; the weights are the bars above.

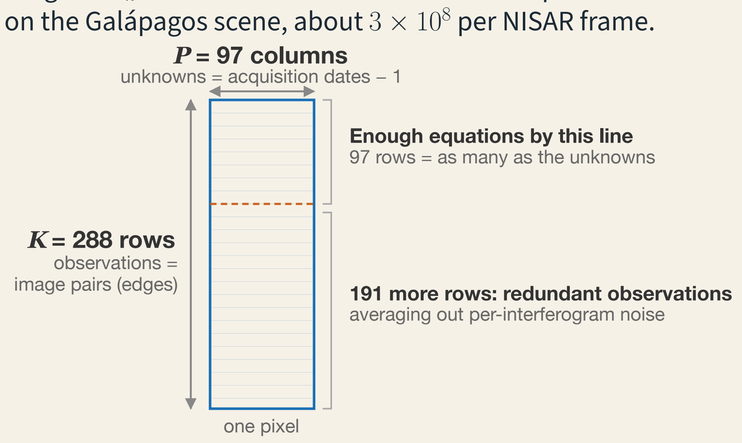
- Weights come from coherence (inverse phase variance, MintPy's default) and differ per pixel, so every pixel repeats this alone: millions of independent small problems.

3 The problem: millions of small least-squares systems

Each pixel k finds the bottom of its own bowl (block 2),

$$\hat{x}_k = \arg\min_x \left\| W_k^{1/2} (G\,x - d_k) \right\|_2^2,$$

with K interferograms (edges) and $P = D - 1$ unknowns for D dates: G ($K \times P$) is shared by every pixel, the phases d_k and weights W_k are not. A scene has $n = 10^5$ to 10^8 pixels: 3.4 million on the Galápagos scene, about 3×10^8 per NISAR frame.

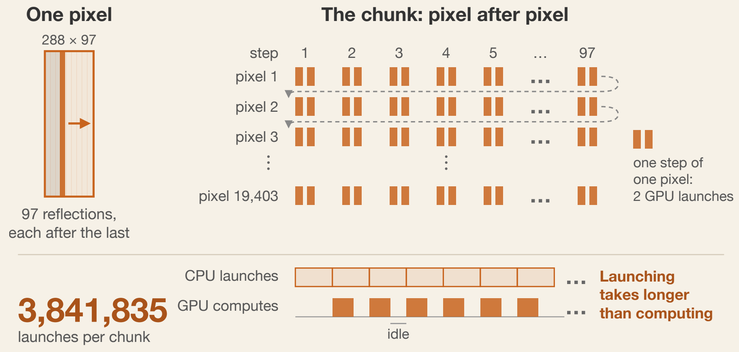


One pixel's system, to scale (Fernandina: $K = 288$, $P = 97$).

4 Method: reformulate for the GPU, don't just port

Porting the CPU solver to a GPU is not enough: the problem has to be recast into a shape the GPU is good at.

A direct port falls short. Handing the pixels to PyTorch's QR least-squares solver on the GPU reached only **1.43**× over the CPU path (Fernandina).

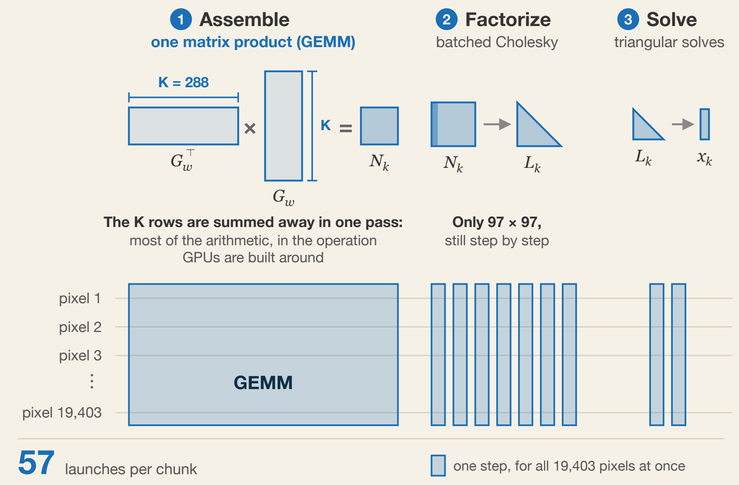


One chunk of 19,403 pixels ($K = 288$, $P = 97$). At this matrix height PyTorch's QR solver works through the pixels one at a time.

Recast around the normal equations,

$$N_k\,x_k = r_k, \qquad N_k = G^\top W_k\,G, \quad r_k = G^\top W_k\,d_k,$$

the same solution in exact arithmetic (its accuracy cost: block 6), in single precision with PyTorch [3] and no custom GPU code.



The same chunk, with $G_w = W_k^{1/2} G$. Missing interferograms enter as zero weights, so every pixel keeps the same shape and the chunk stays one batch.

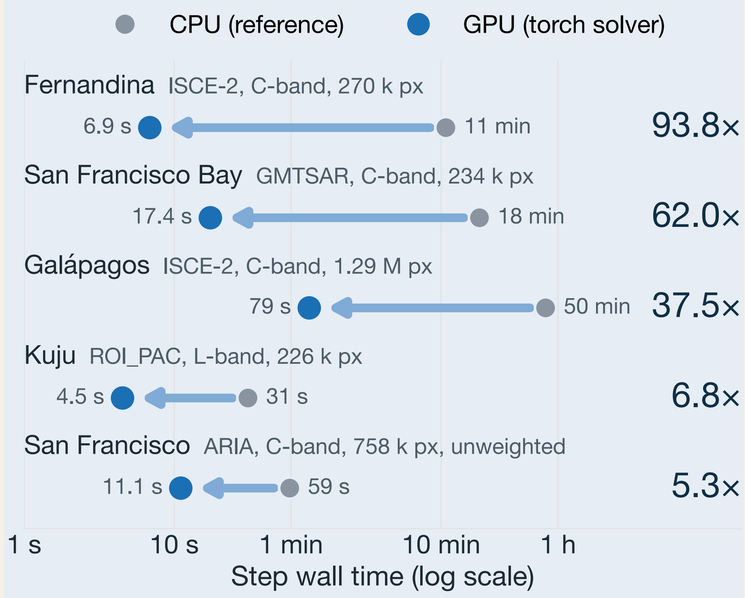
Same GPU, same scene (Fernandina): QR 228 s → Cholesky 13.8 s, **16.5**×. Both ratios in this block are step times reported by MintPy, with interpreter start-up excluded. Gram matrix plus Cholesky is an established strategy for parallel and GPU settings, at the documented cost of squaring the condition number [4, 5]; here only the solution is computed, never a QR factor.

5 Results: speed

Every speedup is measured against the CPU reference path on one consumer GPU (NVIDIA GeForce RTX 5080, 16 GB).

6,169 s → 139 s

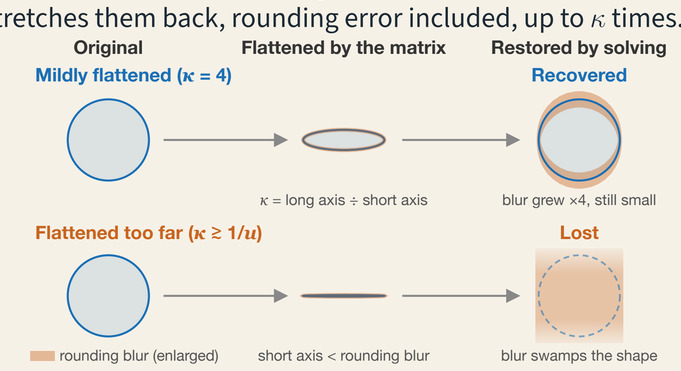
Galápagos scene, 3.4 million pixels × 475 retained interferograms: **44.4**× as timed by MintPy (interpreter start-up excluded), within 7.6 GB of GPU memory



Wall time of the network-inversion step in complete 18-step runs on five public InSAR datasets used to test MintPy: Sentinel-1 (C-band) or ALOS-1 (L-band) stacks over volcanoes and the San Francisco Bay Area, from four processors, all on Zenodo, an open repository for research data. Pixels are those left after masking. On the log axis the arrow length is the speedup. No size dependence is claimed: the ranking does not follow pixel count.

6 The accuracy cost: micrometres, against millimetre signals

A matrix flattens some directions more than others; its *condition number* κ is how flat, the longest axis over the shortest. Solving stretches them back, rounding error included, up to κ times.



The rounding blur is the same size in every direction: $u \approx 6 \times 10^{-8}$ of the longest axis in single precision. A short axis thinner than that ($\kappa \gtrsim 1/u \approx 1.7 \times 10^7$) cannot be restored.

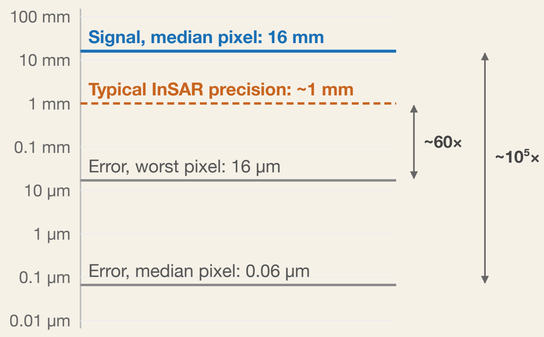
The normal equations flatten twice: $N_k = G_w^\top G_w$ with $G_w = W_k^{1/2} G$, so $\kappa(N_k) = \kappa(G_w)^2$, and single-precision Cholesky fails outright once $\kappa(G_w)$ nears $u^{-1/2} \approx 4.1 \times 10^3$. That is why textbooks solve least squares with QR or SVD, as the CPU path does. The cost is worth paying only if real networks stay clear of that point and the extra error stays below what InSAR resolves, so we measured both.

Scene	κ, unweighted	κ, worst weighted pixel	margin
Fernandina	11.0	323	**12.7**×
Galápagos	12.7	242	**16.9**×
Kuju	28.6	81	**50**×
San Francisco Bay	55.1	721	**5.7**×
San Francisco	51.1	(run unweighted)	**80**×

The five scenes of the speed chart (block 5). Float64 SVD on 2,100 sampled pixels per scene. Margin is the distance from the worst κ of the system actually solved to the breakdown point, 4.1×10^3.

Against the CPU reference, the per-pixel RMS difference of the time series is 0.06 μm for the median pixel and 16 μm for the worst one (Galápagos, 3.4 million pixels). All five scenes pass an agreement gate, RMS difference below 10^{-5} of the largest CPU value, on the final products (velocity and geocoded time series). The few pixels that would break down are low-coherence ones that the standard temporal-coherence mask discards anyway.

Signal vs numerical error (Galápagos)



Against the CPU's SVD-based solver, squaring κ does leave a trace, but at micrometres, buried under InSAR's other error sources.

7 Why it matters: SAR data is multiplying

- NISAR [1] maps nearly all of Earth's land and ice every 12 days.
- Commercial constellations of small SAR satellites add dozens more.
- Taiwan's own SAR satellites, FORMOSAT-9 [6], are planned.

More acquisitions mean longer archives, whose inversion cost grows roughly with the cube of their length, and every new acquisition means inverting the whole network again. Turning that step from hours on a CPU into minutes on one consumer GPU makes routine wide-area monitoring and archive reprocessing practical without an HPC cluster.

As SAR archives multiply, a consumer GPU keeps their time-series analysis within reach of any research group.

The solver is open source and under review for MintPy (pull request #1490 [7]).

Code: PR #1490

Poster PDF (Zenodo)

References

[1] NASA-ISRO SAR (NISAR) Mission Science Users' Handbook, NASA JPL, 2018.
[2] Z. Yunjun, H. Fattahi, F. Amelung, "Small baseline InSAR time series analysis: Unwrapping error correction and noise reduction," *Comput. Geosci.*, 133, 104331, 2019.
[3] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," *NeurIPS*, 32, 2019.
[4] Y. Yamamoto, Y. Nakatsukasa, Y. Yanagisawa, T. Fukaya, "Roundoff error analysis of the CholeskyQR2 algorithm," *ETNA*, 44, 306–326, 2015.
[5] T. Fukaya, Y. Nakatsukasa, Y. Yanagisawa, Y. Yamamoto, "CholeskyQR2: A simple and communication-avoiding algorithm for computing a tall-skinny QR factorization on a large-scale parallel system," METR 2014–37, The University of Tokyo, 2014.
[6] Taiwan Space Agency, FORMOSAT-9, `tasa.org.tw/en-US/missions/detail/FORMOSAT-9`.
[7] S. Sasaki, "Add opt-in torch GPU solver for invert_network," insarlab/MintPy, Pull Request #1490, 2026 (under review).
Benchmark data: InSAR stacks on Zenodo by Z. Yunjun and co-authors, DOI 10.5281/zenodo.⟨n⟩: Fernandina 3952953, Galápagos 4743058 (data of [2]), Kuju 3952917, San Francisco 4265413, San Francisco Bay 15814132.